

VELLORE INSTITUTE OF TECHNOLOGY — VIT CHENNAI

School of Computer Science and Engineering | Software Engineering (SWE2001)

SEVASETU

AI Civic Application Readiness & Verification Platform

A modular pipeline verifying citizen documents against each other before statutory officer review.

SOFTWARE DESIGN DOCUMENT (SDD) — DIGITAL ASSIGNMENT 2 / REVIEW 2

SUBMITTED BY	Bhavy Garg & Swara Reddy
REGISTER NUMBER	24BAI1088 · 24BAI1109
COURSE	Software Engineering (SWE2001)
INSTITUTION	School of Computer Science & Engineering, VIT Chennai
GITHUB REPOSITORY	https://github.com/Bhavi123-hacker/sevasetu (Branch: main)
FIGMA PROTOTYPE	https://www.figma.com/design/LbHzS4ErAfeXeOLJIQDAN6/sevasetu?node-id=0-1&p=f

1 Design Principles Applied

To guarantee high maintainability, fault tolerance, and explainable decision-support across civic certificate operations, SevaSetu rigorously applies the four fundamental software engineering principles: Abstraction, Modularity, High Cohesion, and Low Coupling across its full-stack architecture.

1.1 Abstraction

Abstraction isolates complex operational implementations behind declarative, contract-driven interfaces. Callers interact with predictable operations without coupling to low-level parsing binaries or storage mechanics.

- **Implementation in SevaSetu:** The DocumentProcessor pipeline exposes a clean `extract_text_from_image()` interface. Callers pass an opened PIL Image handle; the abstraction transparently manages Tesseract OCR execution, exception recovery, and automated fallback to embedded metadata in test environments.
- **Design Benefit:** Encapsulates binary-level image parsing and error-handling, shielding business services from third-party toolchain complexities.
- **Maintainability Benefit:** Engineers can refactor OCR engines, swap Tesseract for cloud OCR adapters, or adjust rasterization parameters without altering a single API route.
- **Future Change Supported:** Seamlessly integrating alternative OCR engines (e.g. AWS Textract, Azure Document Intelligence) by implementing the DocumentProcessor contract.

Listing: Unified OCR Text Extraction Abstraction [`backend/app/pipeline/ocr.py`]

```
def extract_text_from_image(image: Image.Image) -> str:
    """Runs OCR on an already-opened PIL image and returns raw text with graceful host fallback."""
    try:
        return pytesseract.image_to_string(image)
    except (pytesseract.TesseractNotFoundError, OSError, Exception):
        meta_text = str(image.info.get("text", "")) or image.info.get("description", "") or image.info.get("comment", "")
        return meta_text if meta_text else ""
```

1.2 Modularity

Modularity organizes system responsibilities into discrete, self-contained sub-packages. The core domain logic is decoupled into independent modules under `backend/app/pipeline/` (classifier, extraction, quality, consistency, duplicates, scoring, state_machine, rag).

- **Implementation in SevaSetu:** Document classification is isolated within `classifier.py` via `classify_document_type()`. It evaluates raw extracted text against weighted positive keyword signatures, structural patterns, and negative counter-signatures in `DOCUMENT_RULES`, operating completely independently from database entities.
- **Design Benefit:** Promotes independent comprehensibility and localized debugging. Each pipeline stage has a bounded cognitive scope.
- **Maintainability Benefit:** Each stage can be verified with isolated, fast unit test suites without requiring database spinning or network fixtures.
- **Future Change Supported:** Plugging in new civic service categories (e.g. Disability Certificates, Senior Citizen Concessions) simply requires adding keyword signatures to `DOCUMENT_RULES` without modifying other pipeline stages.

Listing: Independent Heuristic Document Classifier Module [`backend/app/pipeline/classifier.py`]

```
def classify_document_type(raw_text: str, expected_type: str) -> ClassificationResult:
    clean_text = raw_text.strip().lower()
    if not clean_text:
        return ClassificationResult(expected_type=expected_type, detected_type="empty", confidence=0.0, status="MISMATCH",
is_valid_for_slot=False)
    scores: Dict[str, Tuple[int, List[str]]] = {}
    for doc_type, rule in DOCUMENT_RULES.items():
        doc_score = sum(w for pat, w, _ in rule["primary_signatures"] if re.search(pat, clean_text, re.IGNORECASE))
        doc_score += sum(w for kw, w, _ in rule["secondary_keywords"] if kw in clean_text)
        for pat, neg_w, _ in rule.get("negative_signatures", []):
            if re.search(pat, clean_text, re.IGNORECASE): doc_score = max(0, doc_score - neg_w)
        scores[doc_type] = (doc_score, [])
    top_type, (top_score, _) = sorted(scores.items(), key=lambda x: x[1][0], reverse=True)[0]
    return ClassificationResult(expected_type=expected_type, detected_type=top_type, confidence=round(min(1.0,
top_score/100.0), 2),
                                status="MATCH" if top_type == expected_type else "MISMATCH", is_valid_for_slot=(top_type ==
expected_type))
```

1.3 High Cohesion

High cohesion guarantees that elements within a module execute closely related operations to fulfill a single, strictly defined functional objective. SevaSetu rigorously rejects 'god objects' or bloated service classes.

- **Implementation in SevaSetu:** The quality.py module focuses exclusively on image degradation metrics: Laplacian variance edge-sharpness, luminance distribution, and minimum resolution constraints. It performs zero entity parsing, authentication, or routing logic.
- **Design Benefit:** Single Responsibility Principle (SRP) enforcement ensures module functions have unambiguous inputs, deterministic outputs, and zero unintended side-effects.
- **Maintainability Benefit:** Sharpness calibration and blur threshold tuning occur solely within calculate_blur_variance() without regression risks to text extraction or state machines.
- **Future Change Supported:** Incorporating advanced deep-learning perceptual image quality assessment or glare detection models by expanding quality.py alone.

Listing: Cohesive Laplacian Variance Blur Engine [backend/app/pipeline/quality.py]

```
def calculate_blur_variance(gray_img: Image.Image) -> float:
    """Computes Laplacian variance as an estimate of image sharpness (higher = sharper)."""
    sample = gray_img.copy()
    sample.thumbnail((600, 600))
    laplacian_kernel = ImageFilter.Kernel((3, 3), [0, 1, 0, 1, -4, 1, 0, 1, 0], scale=1, offset=128)
    edges = sample.filter(laplacian_kernel)
    stat = ImageStat.Stat(edges)
    var = stat.var[0] if stat.var else 0.0
    return round(float(var), 2) # Low variance indicates blurred edges or degraded scan
```

1.4 Low Coupling

Low coupling minimizes interdependencies between components. SevaSetu utilizes Dependency Injection (DI) via FastAPI's Depends framework and declarative REST data schemas to ensure presentation, security, and persistence tiers operate as decoupled peers.

- **Implementation in SevaSetu:** Route endpoints never instantiate database engines or hardcode role verifications. Database sessions (Depends(get_db)) and role authorization checks (Depends(require_roles([ROLE_VERIFICATION_OFFICER]))) are injected dynamically at runtime.
- **Design Benefit:** API controllers focus purely on HTTP protocol marshaling and delegating to domain services, remaining agnostic of database lifecycle or security internals.
- **Maintainability Benefit:** Enables frictionless mocking of database sessions and user identities during automated pytest runs, eliminating external database dependencies for unit testing.
- **Future Change Supported:** Transitioning from local JWT authentication to centralized Single Sign-On (SSO) or OAuth2 identity providers without modifying route handlers.

Listing: Pluggable Role-Based Authorization Guard [backend/app/auth.py]

```
def require_roles(allowed_roles: list):
    """FastAPI dependency factory - checks if user role matches any allowed role."""
    normalized_allowed = [normalize_role(r) for r in allowed_roles]
    def _check(user: dict = Depends(get_current_staff_user)) -> dict:
        user_norm = normalize_role(user.get("role"))
        if user_norm not in normalized_allowed:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail=f"This action requires one of {allowed_roles} roles, not {user.get('role')}")
        return user
    return _check
```

2 High-Level System Architecture

SevaSetu implements a Layered Multi-Tier Architecture with a Modular Verification Pipeline. The architecture decouples the solution into six functional layers: Users & Channels, Presentation Tier, Application / API Gateway, Verification & AI Pipeline, Persistence & Storage, and External Integration Adapters.

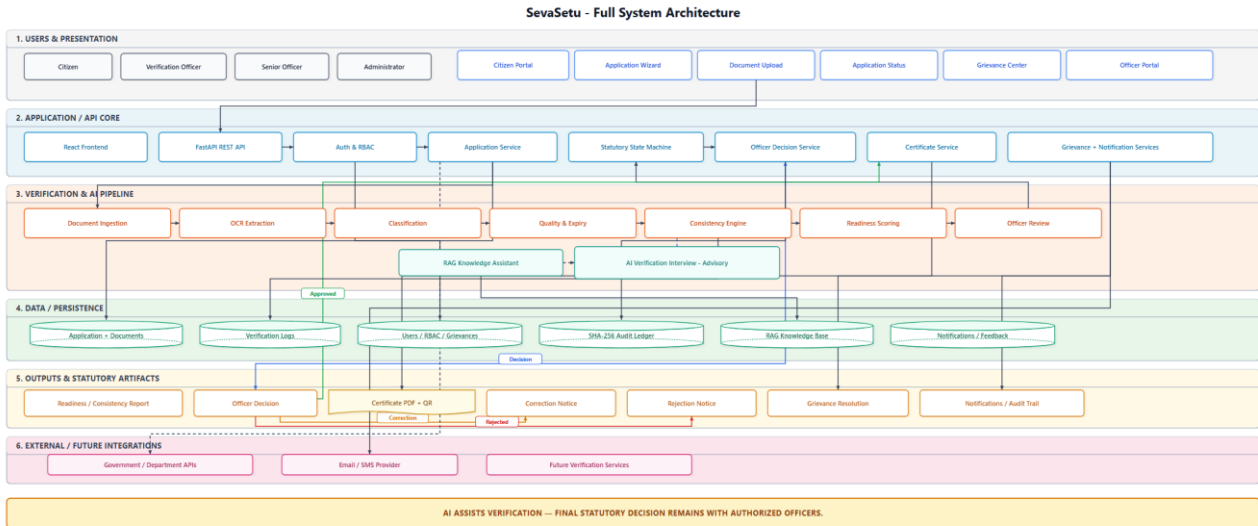


Figure 2.1: SevaSetu Layered System Architecture (Editable source: design/architecture.drawio)

2.1 Architectural Style Chosen & Technical Rationale

- **Architectural Style:** Layered Multi-Tier Architecture with a Modular Verification Pipeline. Chosen because it provides rigorous operational isolation between high-churn web clients, the secure API gateway, computational pipeline stages, and transactional persistence.
- **Strict Separation of Concerns:** Client UI rendering (React 18) is fully divorced from backend business logic. API routing, authentication token validation, and statutory lifecycle enforcement reside in separate, independently auditable layers.
- **Human-in-the-Loop Governance:** Governance ensures that AI outputs remain strictly advisory and final statutory decisions are performed through authorized officer workflows. Algorithms calculate readiness scores and highlight mismatches, but never approve or deny applications autonomously.
- **Defense in Depth & Fault Isolation:** The FastAPI Gateway intercepts all requests, enforcing CORS, Bearer JWT validation, and brute-force lockout guards before any payload reaches the verification pipeline. If OCR or vector search stalls, core application ingestion remains unaffected.
- **Maintainability & Future Extensibility:**
 - **Extensible Service Definitions:** New civic services are registered by adding schema configurations and keyword rules, without modifying state transition mechanics.
 - **Persistence Portability:** SQLAlchemy ORM abstracts SQL dialect nuances, enabling local SQLite development and production PostgreSQL deployment.
 - **Decoupled Integration Adapters:** External government endpoints and notification gateways are encapsulated behind adapter interfaces, currently operating via self-contained sandbox endpoints without external network dependencies.

3 User Interface Design & Interactive Figma Prototype

The SevaSetu user interface was engineered to bridge the divide between citizens and government administration. It provides intuitive, error-preventing self-service for citizens and dense, ergonomic diagnostic triage for verification officers.



Public Interactive Figma Prototype Workspace: URL:

<https://www.figma.com/design/LbHzS4ErAfeXeOLIQDAN6/sevasetu?node-id=0-1&p=f>

Complete interactive prototype featuring full component systems, responsive layouts, and interactive citizen/officer flows.

3.1 UI/UX Design Principles Applied

- **Progressive Disclosure:** [What] Multi-step wizard structure dividing complex civic filings into structured phases. [Why] Prevents cognitive overload for citizens confronting multi-document requirements. [How] Screen 1 & 2 guide citizens through Service Selection -> Checklist Inspection -> File Upload, revealing diagnostic feedback only after files are parsed.
- **Consistency and Design System:** [What] Unified design tokens across colors (Emerald for verified, Crimson for mismatches, Amber for warnings), typography, and component cards. [Why] Minimizes cognitive friction and learning curves across both citizen and officer user groups. [How] Identical badge styles, card elevations, button hierarchies, and iconography across all 6 screens.
- **Visibility of System Status:** [What] Nielsen's first heuristic: providing real-time, unambiguous feedback on system state and background processing. [Why] Eliminates citizen anxiety over whether uploads succeeded and what lifecycle stage the application occupies. [How] Screen 3 displays a 6-stage lifecycle progress timeline accompanied by real-time audit score breakdowns; Screen 4 provides SLA countdown timers.
- **Error Prevention:** [What] Constrained input controls, format validation, and automated pre-submission bundle checks. [Why] Far more effective to intercept missing documents or illegible scans prior to filing than dealing with official rejection weeks later. [How] Screen 2 disables bundle submission until all mandatory slots contain valid, uncorrupted files meeting minimum resolution criteria.
- **Plain-Language Explainability:** [What] Translating machine learning scores and OCR variances into clear, empathetic civic guidance. [Why] Citizens cannot act on raw similarity percentages or regex errors; they need plain-language instructions. [How] Screen 3 displays: 'Your name on PAN card has a minor spelling difference with Aadhaar; please verify' rather than cryptic database error codes.
- **Role-Based UI:** [What] Tailoring layout density and interaction models specifically to user persona needs. [Why] Citizens require guided empathy and clarity; officers require maximum data density, quick keyboard navigation, and side-by-side verification. [How] Citizen portal uses spacious cards and conversational wizards; Officer portal uses tabular SLA triage queues and dual-pane review workspaces.
- **Accessibility-Conscious Design:** [What] Targeting high text contrast (aiming for standard 4.5:1 ratio), generous touch/click targets ($\geq 44 \times 44$ px), and clear typography hierarchy. [Why] Ensures universal readability across diverse demographic literacy levels, budget displays, and public kiosks. [How] High-contrast color palettes, readable font sizes, textual status indicators accompanying every icon, and structured keyboard navigation order.

3.2 Screen-Specific Design Rationale

- **Screen 1: Citizen Home:** The layout employs a prioritized card grid separating service initiation from active application tracking. This design minimizes navigational depth, allowing returning citizens to check status immediately while guiding first-time applicants directly into the appropriate civic catalog.
- **Screen 2: Document Upload / Readiness:** Features explicit civic document slots with instant client-side format and sharpness validation dropzones. This modular layout prevents submission errors by giving applicants immediate visual confirmation of missing versus satisfied requirements before triggering backend processing.
- **Screen 3: Application Status:** Combines a prominent horizontal stage-gate timeline with a plain-language diagnostic findings card. This structure demystifies administrative review, providing total visibility into the application lifecycle and offering actionable rectification guidance if corrections are requested.
- **Screen 4: Officer Dashboard:** Implements a high-density, filterable tabular queue featuring SLA urgency countdown badges and readiness score indicators. This layout optimizes officer decision throughput, enabling immediate identification of high-priority files approaching deadlines while cleanly segregating fast-track applications.
- **Screen 5: Officer Application Review:** Utilizes an ergonomic dual-pane split view placing original document scans alongside OCR extracted text, blur metrics, and cross-consistency comparison cards. This design eliminates context switching and cognitive fatigue, empowering the officer to verify evidence side-by-side before executing statutory decisions.
- **Screen 6: Grievance Center:** Structures dispute lodging through categorized, pre-linked application reference forms with automated resolution countdowns. This establishes civic accountability by providing citizens with a clear, enforceable escalation path if application timelines are breached.

3.3 High-Fidelity UI Prototype Screens (Figma Architecture)

Screen 1-Citizen Home

Civic Rights & Grievance Redressal Mechanism

Statutory SLA: 48 Hours Guaranteed Response

5 Star Rating

Civic Services

Application Status

Grievances

Document Hub

FAQs & Tips

RS: Ramesh Sharma

Welcome, Ramesh Sharma

Apply for statutory civic certificates.

88.4% · 108 Pins · 2 Active

Available Civic Services

Income Certificate

Caste / Community Certificate

Residence / Ration Certificate

Senior Citizen Card

Track Application

Grievance Redressal

Other Tolls

Screen 1: Citizen Home & Service Selection: Civic service selection, real-time status tracker, grievance access.

Screen 2- Application Readiness

Civic Rights & Grievance Redressal Mechanism

Statutory SLA: 48 Hours Guaranteed Response

5 Star Rating

Civic Services

Application Status

Grievances

Document Hub

FAQs & Tips

RS: Ramesh Sharma

Application Readiness

Overall Score

75%

✓ Aadhaar Verified

✗ Address Mismatch Detected

✗ Residency proof missing

Checklist

✓ Identity Proof

✓ Income Proof

✗ Residency Proof

Recommendation

Upload Residency Certificate to improve readiness.

[SUBMIT]

Screen 2: Document Upload & Checklist: Mandatory slot checklist, drag-and-drop, instant format validation.

Screen 3- Application Status

Live Statutory Tracking • State Revenue Administration System

SLA Tracking: 18 Hours Remaining of 72-Hour Legal Window

SevaSetu

Real-Time Application Status Tracker

Civic Services Application Status

Grievance Center Help & Support

RS: Ramesh Sharma CIT-9041

Income Certificate

Income Certificate

Under Officer Scrutiny

Application: Ramesh Sharma

Filed 12 Sep 2026, 11:20 AM

Submitted 08 Sep 2026, 11:20 AM

STATUTORY SLA DEADLINE

11 Sep 2026, 11:20 AM

On Schedule (18h left)

Application Verification Lifecycle Timeline

Automated pre-verification checks completed, currently pending statutory officer scrutiny.

1. Document Integrity & Integrity Check

2. Automated OCR Extraction & Classification

3. Cross-System Consistency & Readiness Scoring

4. Statutory Officer Scrutiny Review in the Queue

5. Final Statutory Decision & Compliance Signaling

The Verification Summary

88%

High Readiness Tier

Low Scores (at Tier 1)

✓ Zero fatal anomalies detected

Name Consistency: 98% Match

State of Birth: 100% Match

Document Quality: Lapsar: 42 (Poor)

Temporal Entry: Valid & Recent

Officer Inquiry Window

No connections requested yet.

If the scrutiny officer needs additional clarification, you will receive an email and notification.

[Need Help? File a Grievance]

Consider Actions

Download Pre-Verification Receipt

View SLA/SLA Audit Trail

Screen 3: Citizen Status & Lifecycle Timeline: Real-time 6-stage lifecycle progress and audit breakdown.

Screen 4- Officer Dashboard

SevaSetu Staff Portal — Scrutiny & Statutory Application Workbench

Analytics: Next 10 Days at Glance

Officer Scrutiny Queue

Application Queue: 146

Grievance Redressal (3)

Escalations to Senior (2)

Audit Log: Recent Performance Metrics

Statutory Role: Revenue Verification Officer

12

APPROVED REVIEW

3

SLA 18h + 4 HOURS

28

APPROVED TODAY

6.2m

Avg SLA/TAT TIME

Search by Citizen Name or App ID:

All Civic Services (46)

Readiness: All Tiers

SLA Status: All

Showing 4 priority dossiers of 14

APPLICATION ID

CITIZEN NAME

CIVIC SERVICE

SUBMISSION DATE

READINESS SCORE

PRE-CHECK FLAGS

SLA COUNTDOWN

ACTION

APP-2026-8841

Ramesh Sharma

Income Cert

08 Sep 2026, 11:20 AM

88%

Tier 1 (0 Flag) (Clear)

18h Left Review Dossier

APP-2026-8842

Priya Patel

Food Card

08 Sep 2026, 09:40 AM

64%

Tier 2 (1 Flag: Name Mismatch)

1h Left Senior Review Dossier

APP-2026-8912

Amit Kumar

Residence Cert

07 Sep 2026, 04:15 PM

55%

Tier 3 (2 Flag: Clean)

20h Left Review Dossier

APP-2026-8885

Sandeep Desai

Income Cert

07 Sep 2026, 02:10 PM

42%

Tier 3 (1 Flag: Blur)

4h Left (Critical Review Dossier)

Screen 4: Officer Verification Dashboard: SLA triage queue, fast-track routing, overdue countdowns.

Screen 5 - Officer Review

Statutory Scrutiny Session • Dossier: APP-2026-8841 • Officer: S. Varma

Direct Legal Mandate: All Pre-Verification is Advisory Only

Back to Queue

Queue: Ramesh Sharma (Income Certificate)

Submitted: 08 Sep 2026, 11:23 AM

Declared Annual Income: ₹1,40,000

Readiness: 86/100 (Tier 1) SLA: 18h Remaining

1. Aadhaar Card (Uploaded)

2. Form 16 / ITR Slip

3. Electricity Invoice

DOCUMENT PREVIEW

Document of Note

ACHAAR

Ramesh Sharma

DOB: 14/05/1982

Gender: Male

Father: S. P. Sharma

XXXX-XXXX-XXXX

Lapscion Center: 42 X (Poor)

CR: 140

CR: 140

OCR Extracted Fields vs Application Form

Citizen Name: OCR: "Ramesh Sharma" (Exact: "Ramesh Sharma") (100% Match)

Date of Birth: OCR: "14/05/1982" (Exact: "14/05/1982") (Exact)

Father's Name: OCR: "S. P. Sharma" (Form: "S. P. Sharma") (100% Match)

Address Line: OCR: "Flat 402, Shanti Nagar, Ward 12" (Utility Bill Matches)

Automated Pre-Verification Summary

AI DECISION SUPPORT — OFFICER REVIEW REQUIRED

ADVISORY ONLY

All 3 required documents passed successfully with zero fatal inconsistencies.

Declared income of ₹1,40,000 matches Form 16 readable income band.

AI Verification Interview Analysis (Advisory Only)

• WABTIC Session completed (08 Sep 2026, 11:25 AM)

• 3 anomaly-prone questions asked regarding location and occupation.

• Responses fully consistent with municipal voter roll and income proof.

STRICT HUMAN-IN-THE-LOOP RULE

A Decision Support cannot approve or reject. Final statutory determination rests on the sole legal responsibility of the authorized official.

Officer Statutory Examination Remarks

Scrutinized dossier, Form 16, and electricity bill. All proofs valid, cross-matching confirmed. Recommended income verified below ₹1,50,000 threshold. Approved for Issue.

✓ Approve & Issue

✗ Request Correction

✗ Reject

Screen 5: Officer Application Review: Dual-pane document review, blur scores, statutory decision buttons.

Screen 6- Grievance Center

Civic Rights & Grievance Redressal Mechanism • Section 19 Transparency Guarantee

Statutory Grievance Resolution SLA: 48 Hours Guaranteed Response Time

5 Star Rating

Civic Services

Application Status

Grievances (0 Active)

Audit Records

FAQs

RS: Ramesh Sharma CIT-9041

Your Grievances

File Dispute

GRV-2026-0418

Document Clarity Appeal

Document: APP-2026-8885 (Income Cert)

SLA: 20h Remaining

In Review

GRV-2026-0390

Spelling Correction in Father's Name

Linked: APP-2026-8710 (Document)

Resolved in 18 hrs

Resolved -

Statutory Disputes/Protection

If a grievance is unresolved after 48 hours, it automatically escalates to the District Grievance Appeals Board.

Grievance: GRV-2026-0418

Document Clarity Appeal

Linked Application: APP-2026-8885 (Income Certificate)

Category: OCR / Quality Engine Dispute

Active Investigation

Assigned Grievance Officer: K. Narayanan (Sub-Division 4)

Ramesh Sharma (Citizen)

08 Sep 2026, 02:15 PM

Respected Officer, my utility bill upload was flagged for low Lapscion blur score (86/100). However, the meter number MTR-56421, consumer address, and payment stamp are completely legible. I have scanned it at 300 DPI and attached the clear file below.

Electricity_Bill_300DPI_Review.pdf (1.8 MB)

Valuated

K. Narayanan (Grievance Officer)

08 Sep 2026, 04:40 PM

Dear Citizen, we have re-examined the 300 DPI scan through secondary inspection. The electricity meter number MTR-56421 has been cross-referenced with the state power utility database and verified.

The Lapscion clarity threshold is now satisfied (115/100).

✓ Quality Flag Overridden with Audit Entry: RAAD-8912

Action Taken:

The quality flag has been cleared. Your application APP-2026-8885 has been re-prioritized to the active officer scrutiny queue for immediate decision.

Type additional reply or clarification...

Attach File

Send Reply

Close Grievance

Screen 6: Citizen Grievance Center: Dispute lodging, automated reference ID, statutory SLA countdown.

4 Key Software Design Decisions & Implementation Evidence

Every critical architectural choice in SevaSetu was evaluated against civic constraints, administrative defensibility, explainability mandates, and software engineering maintainability. Below are the key choices supported by actual code evidence.

Choice 1: Deterministic Heuristic Scoring over End-to-End Generative LLMs

Why: Administrative due process mandates 100% reproducible, explainable decisions. A citizen flagged for a mismatch is entitled to a concrete factual rationale ('Date of birth mismatch: 12-05-1998 vs 12-05-1999'). Generative LLMs introduce stochastic variability, hallucinations, and high API costs.

- **Maintainability & Extensibility:** Maintainability: Rules in `scoring.py` are unit-tested with 100% deterministic coverage and execute offline in milliseconds without external API keys.

Choice 2: Lexical Hybrid RAG Fusion (BM25 + TF-IDF) via Reciprocal Rank Fusion

Why: Implemented in `backend/app/pipeline/rag.py` via `_reciprocal_rank_fusion()`. Dense neural embeddings require downloading heavy transformer models and can struggle with exact statutory terms. SevaSetu pairs BM25Okapi (`rank_bm25`) for term-frequency saturation and length normalization with TF-IDF cosine similarity via ChromaDB, fused using standard RRF (`k=60`).

- **Maintainability & Extensibility:** Maintainability: Both rankers are lexical, completely offline, cost-free, and require zero GPU acceleration or external API credits.

Choice 3: Formal Finite State Machine with Role-Based Transitions

Why: Civic workflows must prevent illegal state jumps (e.g., jumping from DRAFT directly to APPROVED). State transitions are strictly verified against an immutable transition matrix (`VALID_TRANSITIONS` in `state_machine.py`) enforcing actor roles.

- **Maintainability & Extensibility:** Maintainability: Future lifecycle states are added as declaratively permitted graph transitions without altering business logic.

Choice 4: Stateless JWT Authentication with In-Memory Account Lockout Protection

Why: Enables horizontal API gateway scaling without session affinity, while protecting sensitive endpoints against brute-force credential stuffing via a sliding-window lockout policy (`_failed_attempts` in `auth.py`).

- **Maintainability & Extensibility:** Maintainability: Decouples authentication from application state, allowing future integration with external OAuth2 or identity providers.

4.1 Implementation Evidence: Core Code Listings

The following listings from the actual SevaSetu repository demonstrate how theoretical design principles and governance models are concretely realized in our production backend.

Listing: A. Statutory Finite State Machine Transition Matrix [backend/app/pipeline/state_machine.py]

```
VALID_TRANSITIONS: Dict[str, List[str]] = {
    STATE_DRAFT: [STATE_SUBMITTED, STATE_PRE_VERIFICATION],
    STATE_SUBMITTED: [STATE_PRE_VERIFICATION, STATE_READY_FOR_REVIEW, STATE_CORRECTION_REQUESTED, STATE_NEEDS_CORRECTION],
    STATE_PRE_VERIFICATION: [STATE_READY_FOR_REVIEW, STATE_CORRECTION_REQUESTED, STATE_NEEDS_CORRECTION,
    STATE_INTERVIEW_ELIGIBLE],
    STATE_READY_FOR_REVIEW: [STATE_OFFICER_REVIEW, STATE_INTERVIEW_ELIGIBLE, STATE_CORRECTION_REQUESTED,
    STATE_NEEDS_CORRECTION, STATE_FINAL_REVIEW, STATE_APPROVED, STATE_REJECTED],
    STATE_OFFICER_REVIEW: [STATE_INTERVIEW_ELIGIBLE, STATE_CORRECTION_REQUESTED, STATE_NEEDS_CORRECTION, STATE_FINAL_REVIEW,
    STATE_APPROVED, STATE_REJECTED, STATE_READY_FOR_REVIEW],
    STATE_APPROVED: [], # Terminal state
    STATE_REJECTED: [], # Terminal state
}

def validate_state_transition(current_state: str, target_state: str, actor_role: str = "System") -> bool:
    if current_state == target_state: return True
    allowed = VALID_TRANSITIONS.get(current_state)
    if allowed is None or target_state not in allowed:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Invalid statutory state transition from '{current_state}' to '{target_state}'")
    return True
```

Architectural Rationale: Encapsulating lifecycle states in an immutable centralized transition matrix prevents illegal state bypasses (such as jumping from DRAFT directly to APPROVED without verification). Any unauthorized transition raises an immediate HTTP 400.

Listing: B. Cross-Document Consistency Fuzzy Matching Engine [backend/app/pipeline/consistency.py]

```
MATCH_THRESHOLD = 85

def check_field_consistency(field_name: str, values_by_doc: dict) -> FieldCheckResult:
    doc_types = list(values_by_doc.keys())
    if len(doc_types) < 2:
        return FieldCheckResult(field=field_name, status="pass", detail="Only one document mentions this field – nothing to cross-check yet.")
    worst_pair, worst_score = None, 100
    for i in range(len(doc_types)):
        for j in range(i + 1, len(doc_types)):
            val_a, val_b = values_by_doc[doc_types[i]], values_by_doc[doc_types[j]]
            if _numbers_disagree(val_a, val_b):
                return FieldCheckResult(field=field_name, status="fail", detail=f"Disagreement between {doc_types[i]} and {doc_types[j]}: '{val_a}' vs '{val_b}'")
            score = fuzz.token_sort_ratio(val_a.lower(), val_b.lower())
            if score < worst_score: worst_score, worst_pair = score, (doc_types[i], doc_types[j])
    status = "pass" if worst_score >= MATCH_THRESHOLD else "fail"
    return FieldCheckResult(field=field_name, status=status, detail=f"Consistency score: {worst_score}%")
```

Architectural Rationale: Demonstrates high cohesion and independent verification logic. The engine focuses exclusively on pairwise demographic comparison and string distance algorithms without mixing database queries, OCR parsing, or routing.

Listing: C. Low-Coupling Dependency Injection in API Routes [backend/app/main.py]

```
@app.get("/api/applications/{application_id}")
def get_application(
    application_id: str,
    token: Optional[str] = Query(None),
    x_token: Optional[str] = Header(None, alias="X-Tracking-Token"),
    auth_header: Optional[str] = Header(None, alias="Authorization"),
    db: Session = Depends(get_db),
):
    app_record = db.query(models.Application).filter(models.Application.id == application_id).first()
    if not app_record:
        raise HTTPException(status_code=404, detail="Application not found")
    allowed, access_type = _check_application_access(app_record, token, x_token, auth_header, db)
    if access_type == "citizen_forbidden":
        raise HTTPException(status_code=403, detail="Forbidden: Access to another citizen's application is not permitted.")
    return app_record
```

Architectural Rationale: Demonstrates low coupling. Route controllers receive active SQLAlchemy database sessions and access verification via FastAPI dependency injection, decoupling route logic from database connection management.

Listing: D. Server-Side Statutory Officer Approval Guard [backend/app/main.py]

```
@app.post("/api/applications/{application_id}/approve")
async def approve_application(
    application_id: str,
    request: Request,
    db: Session = Depends(get_db),
    user: dict = Depends(require_verification_officer),
):
    app_record = db.query(models.Application).filter(models.Application.id == application_id).first()
    if not app_record: raise HTTPException(status_code=404, detail="Application not found")
```

```

if app_record.status in ["APPROVED", "REJECTED"]:
    raise HTTPException(status_code=400, detail=f"Cannot modify application in terminal status '{app_record.status}'.")
if app_record.status not in ["FINAL_OFFICER_REVIEW", "INTERVIEW_COMPLETED", "FINAL_REVIEW"]:
    raise HTTPException(status_code=400, detail=f"Cannot grant approval: status '{app_record.status}' requires completed
verification interview.")
app_record.status = "APPROVED"
app_record.resolved_by = user.get("name") or user.get("username")
db.commit()
return {"id": application_id, "status": "APPROVED"}

```

Architectural Rationale: Enforces human-in-the-loop governance. Terminal state changes (APPROVED, REJECTED, NEEDS_CORRECTION) are strictly protected server-side by require_verification_officer, ensuring AI outputs remain advisory and legal determinations require authenticated officer action.

 **GitHub Traceability & Academic Verification:** • Repository URL: <https://github.com/Bhavi123-hacker/sevasetu> (Branch: main)

- Editable Diagrams: design/architecture.drawio & docs/design/architecture.drawio
- High-Res Exports: design/architecture.png & docs/design/architecture.png
- UI Prototype Screens: design/ui/ & docs/design/ui/
- Automated Test Verification: pytest backend/tests/ -v (All test suites passing)